Software Test Document

1. Document Control

Project Name: Tfl-What-If-Simulation
Document title: Software Test Document
Version: 0.1
Author: Sattyaj Paul
Date: 29 April 2026
Status: In Progress

2. Introduction

2.1 Purpose

This document outlines the testing strategy, testing processes, implemented tests and planned testing activities for the project. The purpose of this document is to ensure the reliability, correctness, and stability of the system throughout development.

2.2 Scope

This testing document applies to all implemented project features derived from approved user stories, including graph construction, pathfinding functionality, database validation, frontend interactions, and backend communication.

2.3 References

- Project User Stories
- GitLab Repository
- Module Specification
- Neo4j Documentation
- Jest Documentation

3. Test Strategy

3.1 Testing Objectives

The testing process aims to:

- Verify correctness of implemented functionality
- Detect defects early during development
- Ensure reliable route generation and graph handling
- Validate frontend and backend integration
- Maintain overall software quality throughout the project lifecycle

3.2 Testing Levels

Testing Level	Description
Unit Testing	Tests individual functions, algorithms and components.
Integration Testing	Verifies communication between frontend, backend and database systems.
System Testing	Validates complete end-to-end functionality.
User Acceptance Testing	Confirms implemented features satisfy user story requirements.

3.3 Testing Types

Test Type	Purpose
Functional Testing	Verifies expected system behaviour.
Edge Case Testing	Validates handling of unusual or invalid inputs.
Database Testing	Ensures Neo4j data integrity and connectivity.
Regression Testing	Ensures new changes do not break existing functionality.
Manual UI Testing	Validates user interface functionality and usability.

4. Current Implemented Tests

4.1 Graph Construction Testing

Implemented unit tests validate:

- Graph creation from nodes and edges
- Bidirectional edge creation
- Multiple line handling
- Invalid edge handling
- Data integrity and immutability
- Complex graph structures
- Edge case handling

4.2 Pathfinding Algorithm Testing

Implemented tests validate the Dijkstra pathfinding algorithm under multiple scenarios:

- Shortest path generation
- Closed station handling
- Closed line handling
- Blocked edge routing
- Combined constraints
- Disconnected graph handling
- Zero-weight and large-weight edge handling
- Complex network topologies

4.3 Database Testing

Implemented database tests validate:

- Neo4j database connectivity
- Station node existence
- Graph integrity validation
- Database accessibility during runtime

5. Planned Future Testing

5.1 Integration Testing

Planned integration tests will verify:

- Frontend to backend communication
- API request and response handling
- Route retrieval from the database
- Backend processing correctness

5.2 Manual User Interface Testing

Manual testing will validate:

- User interactions
- Route search workflows
- Error handling behaviour
- Input validation
- Overall usability and responsiveness

5.3 User Acceptance Testing

User acceptance testing will be conducted against approved user stories and acceptance criteria to confirm functional requirements are satisfied.

6. Test Environment

Component	Technology
Programming Language	JavaScript
Testing Framework	Jest
Database	Neo4j
Version Control	GitLab
Runtime Environment	Node.js

7. Test Coverage Areas

Component	Testing Approach
Graph Builder	Unit Testing
Pathfinding Algorithm	Unit Testing
Neo4j Database	Database Testing
Frontend User Interface	Manual Testing

Backend Services	Integration Testing

8. Roles and Responsibilities

Role	Responsibility
Testing Lead	Oversees testing activities, test planning, and quality assurance
Software Development Lead	Oversees implementation and maintains code quality
Project Management Lead	Coordinates tasks, timelines, and project progress
Documentation Lead	Maintains document structure, consistency, and version control

9. Defect Management

Defects will be tracked using GitLab Issues. Each defect report should include:

- Defect description
- Steps to reproduce
- Expected behaviour
- Actual behaviour
- Severity level
- Supporting screenshots or logs where applicable

10. Risks and Mitigation

Risk	Impact	Mitigation
Unclear requirements	High	Early review of user stories and acceptance criteria
Time constraints	Medium	Prioritised testing of critical functionality
Integration failures	Medium	Continuous integration and early integration testing
Database inconsistencies	Medium	Regular graph integrity testing

11. Entry and Exit Criteria

11.1 Entry Criteria

Testing may begin when:

- User stories are approved
- Features are implemented
- Code is committed to the repository
- Required test environments are available

11.2 Exit Criteria

Testing is considered complete when:

- All planned tests have been executed
- Critical defects are resolved
- Acceptance criteria are satisfied
- Test results are documented

12. Test Deliverables

The following testing deliverables will be produced throughout the project:

- Software Test Document
- Unit Test Suites
- Integration Test Results
- Manual Testing Reports
- Defect Reports
- Final Test Summary Report